

- title: "Essentios – MVP Développement (v0.1)"author: "Blueprint D v"date: "Novembre 2025"
-  Roadmap de D veloppement – MVP Essentios (14 jours)
 -  Vue synth tique (14 jours)
 -  Semaine 1 – Fondations & IA locale (Jour 1–7)
 -  Objectif :
 -   tapes de code :
 -  Semaine 2 – S curit , Signature et Transparence (Jour 8–14)
 -  Objectif :
 -   tapes de code :
 -  Structure cible du projet
 -  Objectif fin de MVP (Jour 14)

title: "Essentios – MVP D veloppement (v0.1)"
author: "Blueprint D v" date: "Novembre 2025"

 Roadmap de D veloppement – MVP
Essentios (14 jours)

Essentios est un **assistant LLM local** pour l  cosyst me **Cosmos**. Ce document r sume les **phases de d veloppement**, les **modules   coder**, et les **fichiers cl s** du projet.

 Vue synth tique (14 jours)

Jour	Module / Composant	Description principale	Langage / Dossier
1	Setup projet	Init repo Git, structure <code>cli/</code> (Python) & <code>contracts/</code> (Rust CosmWasm), config Juno local.	—
2	<code>contracts/witness_journal</code>	Contrat CosmWasm minimal : <code>InstantiateMsg</code> , <code>AppendLog</code> , <code>GetLogs</code> .	Rust
3	<code>cli/core/client.py</code>	Client RPC/GRPC Cosmos via <code>cosmpy</code> , test de connexion.	Python
4	<code>cli/core/parser.py</code>	Parsing du langage naturel → JSON <code>intent_schema.json</code> .	Python
5	<code>cli/llm/engine.py</code>	Int�gration Llama2 (via <code>ollama</code>) pour conversion NL → Tx JSON.	Python

Jour	Module / Composant	Description principale	Langage / Dossier
6	<code>cli/core/tx.py</code>	Génération de transactions Cosmos (<code>MsgSend</code> , <code>MsgDelegate</code>).	Python
7	<code>cli/core/simulate.py</code>	Simulation “dry-run” RPC + gestion des erreurs.	Python
8	<code>cli/core/confirm.py</code>	Interface confirmation [Y/N] + affichage diff Tx.	Python
9	<code>cli/core/execute.py</code>	Signature + broadcast via keyring local.	Python
10	<code>cli/core/log.py</code>	Appel CosmWasm : journalisation des Tx sur <code>witness_journal</code> .	Python
11	<code>cli/memory/ipfs.py</code>	Persistence Off-chain sur IPFS local.	Python
12	<code>cli/ui/summary.py</code>	Résumé textuel des actions réalisées (log de session).	Python
13	Tests intégrés	Scénario complet : “Stake 10 ATOM chez X” → simulation → broadcast → log.	—
14	Documentation / Audit	README technique, schéma modules, nettoyage code.	—

Semaine 1 – Fondations & IA locale (Jour 1–7)

Objectif :

Mettre en place les **bases logicielles** : structure du repo, LLM local, parsing et simulation.

Étapes de code :

Étape	Fichier(s)	Description	Notes
1. Setup projet	<code>cli/</code> , <code>contracts/</code>	Initialiser environnement Python + Rust.	<code>python -m venv venv</code> , <code>cargo init</code>
2. Contrat journal	<code>contracts/witness_journal/src/</code>	Définir structure <code>LogEntry</code> , exécution <code>AppendLog</code> .	Utiliser CosmWasm 1.2

Étape	Fichier(s)	Description	Notes
3. Client RPC	<code>cli/core/client.py</code>	Wrapper <code>cosmpy</code> pour <code>get_balance</code> , <code>simulate_tx</code> .	Appel <code>http://localhost:26657</code>
4. Parsing NL	<code>cli/core/parser.py</code>	Traduire phrase → JSON avec schéma d'intention.	Exemple : "Stake 10 ATOM chez X"
5. LLM Local	<code>cli/llm/engine.py</code>	Implémenter fonction <code>generate_tx(intent)</code> via <code>ollama</code> .	Modèle : <code>llama2:7b</code>
6. Transaction Builder	<code>cli/core/tx.py</code>	Construire <code>MsgSend</code> , <code>MsgDelegate</code> .	Import <code>cosmpy.tx</code>
7. Simulation	<code>cli/core/simulate.py</code>	Simuler transaction (dry-run RPC).	Vérifie gas & validité JSON

Semaine 2 – Sécurité, Signature et Transparence (Jour 8–14)

Objectif :

Relier le cœur IA aux mécanismes on-chain et à la mémoire persistante.

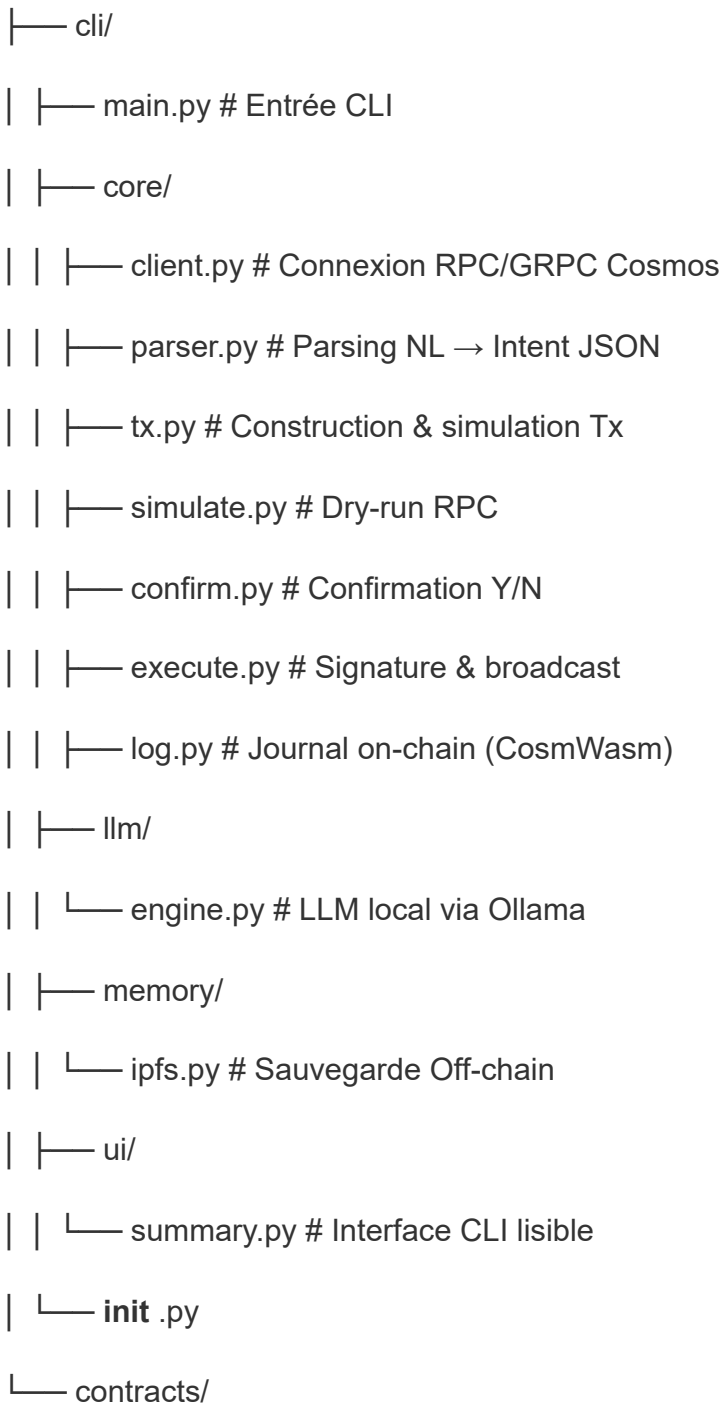
Étapes de code :

Étape	Fichier(s)	Description	Notes
8. Confirmation	<code>cli/core/confirm.py</code>	Prompt Y/N avant signature.	<code>click.confirm("Signer ?")</code>
9. Signature & Broadcast	<code>cli/core/execute.py</code>	Envoi via <code>keyring</code> ou <code>cosmpy.broadcast_tx</code> .	Ledger ou local keyring
10. Journal On-chain	<code>cli/core/log.py</code>	Envoi du hash/TxID au contrat CosmWasm.	<code>AppendLog{tx_id, msg}</code>
11. Mémoire Off-chain	<code>cli/memory/ipfs.py</code>	Envoi du log complet vers IPFS + retour hash.	<code>ipfshttpclient</code>

Étape	Fichier(s)	Description	Notes
12. Interface utilisateur	<code>cli/ui/summary.py</code>	Afficher résumé NL des actions exécutées.	Historique utilisateur
13. Tests	<code>tests/test_end_to_end.py</code>	Exécution complète d'un scénario.	Pytest + mock RPC
14. Documentation	<code>/docs</code>	Générer README, schéma et API CLI.	<code>mkdocs</code> ou <code>mdbook</code>

Structure cible du projet

essentios/



└─ witness_journal/

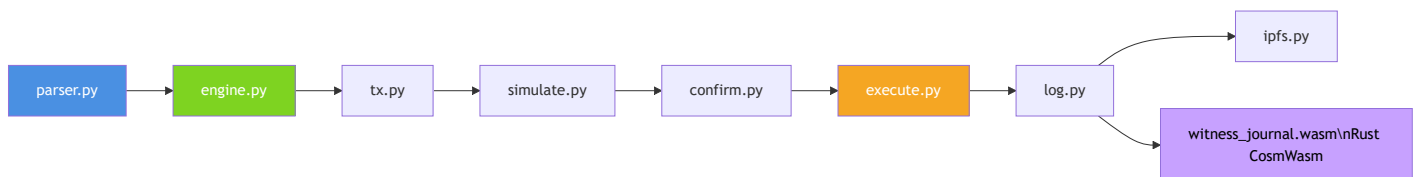
└─ src/

| └─ contract.rs

| └─ msg.rs

| └─ state.rs

└─ Cargo.toml



Flux clair :

1. Langage naturel → parsing → JSON → simulation
2. Confirmation → signature → broadcast
3. Logging → IPFS + Contrat CosmWasm

Objectif fin de MVP (Jour 14)

-  CLI fonctionnelle (**essentios**)
-  LLM local (Llama2 7B)
-  Contrat CosmWasm **witness_journal.wasm**
-  Transaction complète (stake / DAO)
-  Log on-chain + mémoire off-chain
-  Documentation technique prête

Essentios — L'agent LLM on-chain pour Cosmos : *local, transparent, et sous contrôle humain*.